

MECHATRONICS AND IOT

ASSIGNMENT REPORT - 3

TITLE:

Design and develop a Smart Inventory Management System for Construction Sites. The system tracks materials and inventory movement to improve resource utilization and site management efficiency.

DONE BY ,

HARI KRISHNA JAGADEESH-24MER011

HARISH DHARMALINGAM – 24MER014

RANJITH KUMAR P – 24MER048

RISHI KESAV A – 24MER049

VENKATRAMANAN B – 24MER061

Project Overview:

The Smart Inventory Management System is an IoT-based system designed to monitor and manage construction materials efficiently. Construction sites handle large quantities of materials such as cement, steel, bricks, sand and tools, making manual inventory tracking difficult and prone to errors.

The proposed system uses sensors and a microcontroller to monitor the quantity and movement of materials in real time. Inventory information can be displayed locally and stored digitally, allowing site personnel to track available stock, material usage and movement.

The system helps reduce material wastage, prevent stock shortages and improve overall resource utilization and construction site management efficiency.

Problem Statement:

Construction sites handle large quantities of materials such as cement, steel, bricks, sand and tools, making inventory management a challenging task. Traditional manual methods of recording material quantities and movement are time-consuming and prone to errors.

Lack of real-time inventory information can lead to material shortages, overstocking, wastage, misplacement and inefficient resource utilization. It can also make it difficult for site managers to track when materials are received, stored or consumed.

Therefore, there is a need for a smart and automated inventory management system that can continuously track material quantities and movement, maintain accurate inventory records and provide timely information to improve resource utilization, reduce material wastage and enhance construction site management efficiency.

Proposed Solution:

The proposed system is a smart inventory management solution designed to automatically monitor and track construction materials. The system uses an ESP32 microcontroller, sensors and identification modules to record material quantity and movement in real time.

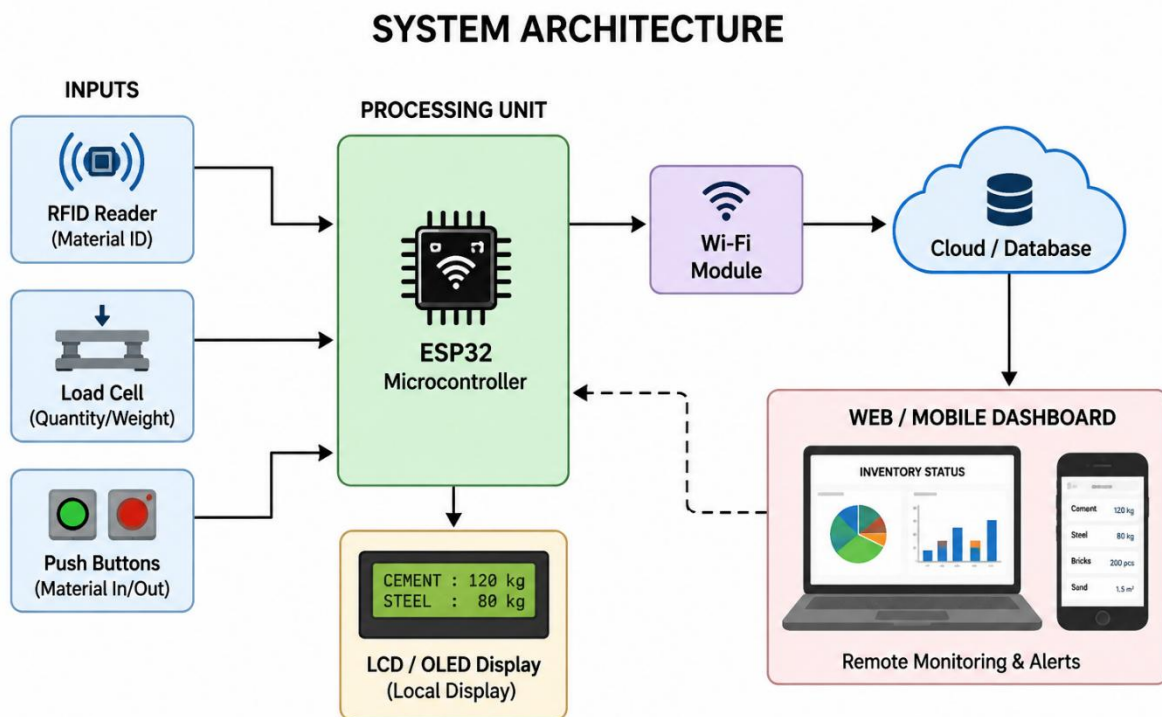
RFID tags and an RFID reader can be used to identify materials entering or leaving the storage area, while a load cell with a load sensor module can measure the quantity or

weight of selected materials. The ESP32 processes the collected information and updates the inventory status.

The current inventory information can be displayed on an OLED/LCD display, while Wi-Fi connectivity can be used to transmit the data to a cloud platform or database for remote monitoring. When the available quantity falls below a predefined minimum level, the system can generate a low-stock alert.

The proposed solution provides a low-cost, automated and real-time inventory monitoring system that helps reduce material wastage, prevent stock shortages, track material movement and improve resource utilization at construction sites.

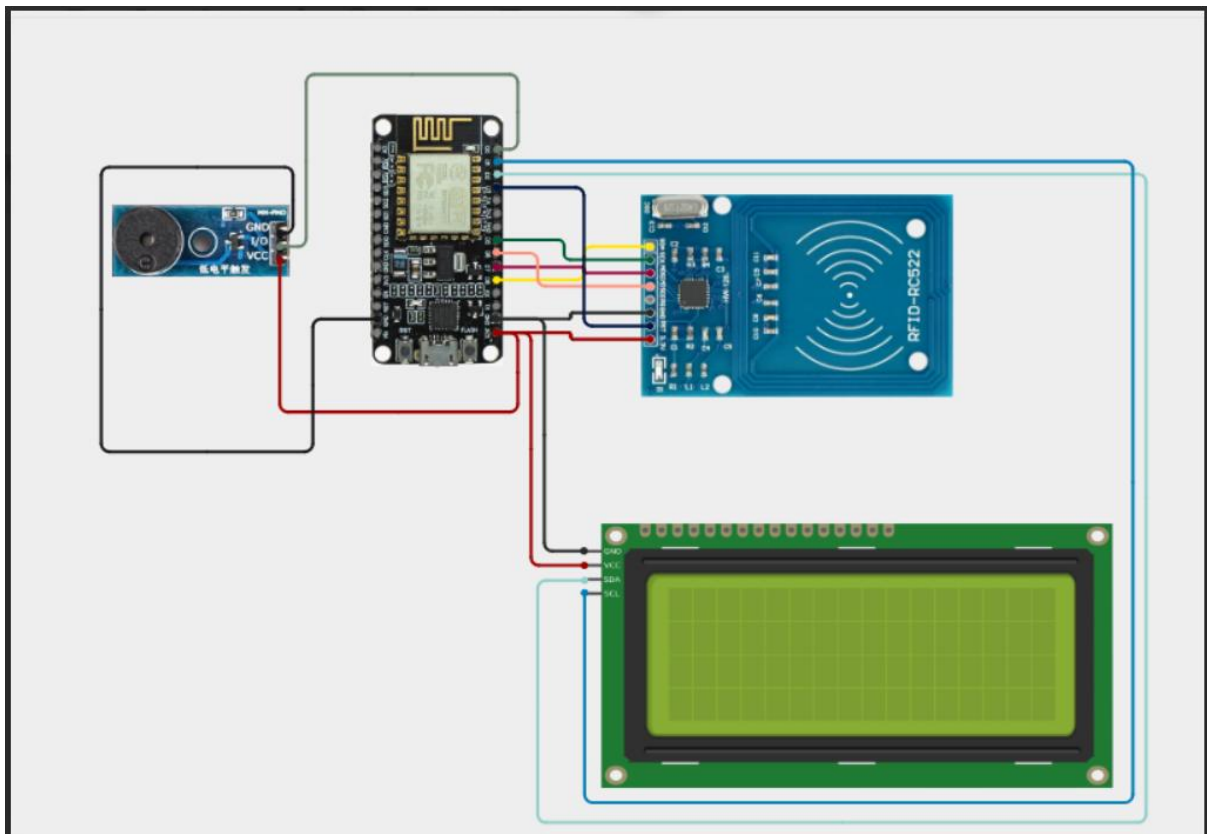
System Architecture:



Circuit Table :

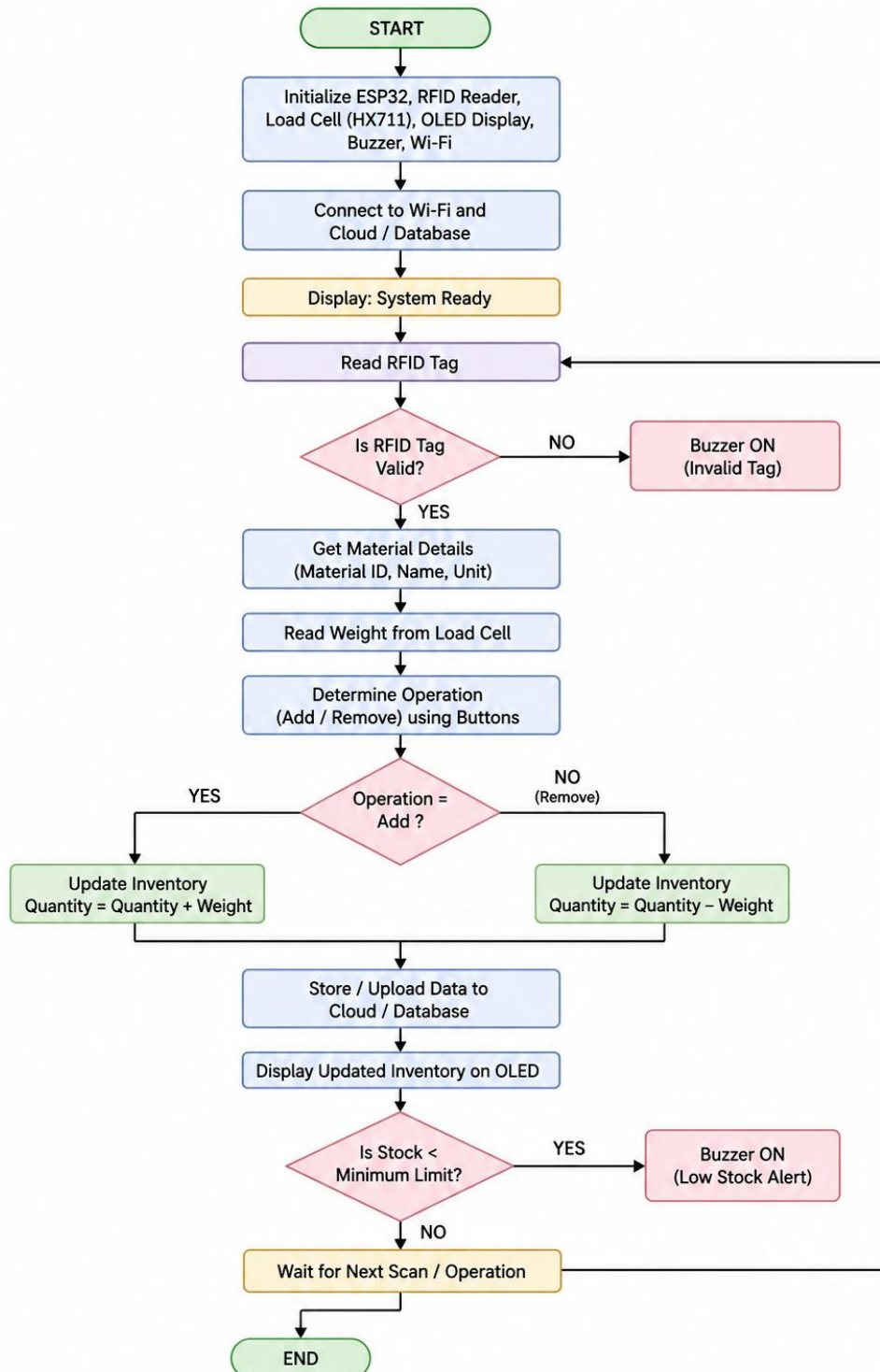
Component	Pin/Terminal	ESP8266 Connection	Purpose
RFID RC522	VCC	3.3V	Power Supply
LCD Display	VCC	3.3V	Safe indication
ESP8266	VIN/USB	5V USB	Hazardous indication
Buzzer	Positive (+)	GPIO 14	Alert

Circuit Design:



Algorithm:

ALGORITHM – SMART INVENTORY MANAGEMENT SYSTEM



Coding:

```
#include <SPI.h>

#include <MFRC522.h>

#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#include <ESP8266WiFi.h>

#include <ThingSpeak.h>

// =====

// WIFI SETTINGS

// =====

const char* ssid = "Xiaomi 11i";

const char* password = "12345678";

WiFiClient client;

// =====

// THINGSPEAK SETTINGS

// =====

unsigned long channelID = 3458770;

const char* writeAPIKey = "PH9Z3KRMKRAGSKBZ";

// =====

// RC522 RFID - ESP8266 GPIO PINS

// =====

#define SS_PIN 15    // D8

#define RST_PIN 0    // D3

MFRC522 mfrc522(SS_PIN, RST_PIN)

// =====

// I2C LCD - ESP8266 GPIO PINS

// =====

#define SDA_PIN 4    // D2

#define SCL_PIN 5    // D1
```

```

LiquidCrystal_I2C lcd(0x27, 16, 2);

// =====

// BUZZER - LOW ACTIVE

// =====

#define BUZZER_PIN 16 // D0

// LOW = Buzzer ON

// HIGH = Buzzer OFF

// =====

// SETUP

// =====

void setup() {

    // -----

    // SERIAL MONITOR

    // -----

    Serial.begin(9600);

    delay(1000);

    Serial.println();

    Serial.println("=====");

    Serial.println("    RFID SYSTEM");

    Serial.println("=====");

    // =====

    // BUZZER SETUP

    // =====

    pinMode(BUZZER_PIN, OUTPUT);

    // LOW ACTIVE BUZZER

    // HIGH = OFF

    digitalWrite(BUZZER_PIN, HIGH);

    // =====

    // START LCD

```

```
// =====  
Wire.begin(SDA_PIN, SCL_PIN);  
lcd.init();  
lcd.backlight();  
lcd.clear();  
lcd.setCursor(0, 0);  
lcd.print("RFID SYSTEM");  
lcd.setCursor(0, 1);  
lcd.print("Starting...");  
  
// =====  
  
// CONNECT TO WIFI  
  
// =====  
  
Serial.println();  
Serial.println("Connecting to WiFi...");  
lcd.clear();  
lcd.setCursor(0, 0);  
lcd.print("Connecting WiFi");  
WiFi.begin(ssid, password);  
while (WiFi.status() != WL_CONNECTED) {  
    delay(500);  
    Serial.print(".");  
}  
Serial.println();  
Serial.println("WiFi Connected");  
Serial.print("IP Address: ");  
Serial.println(WiFi.localIP());  
lcd.clear();  
lcd.setCursor(0, 0);  
lcd.print("WiFi Connected");
```



```

delay(1000);

// =====

// START THINGSPEAK

// =====

ThingSpeak.begin(client);
Serial.println("ThingSpeak Ready");

// =====

// START RFID

// =====

SPI.begin();
mfr522.PCD_Init();
delay(1000);

// =====

// READY

// =====

lcd.clear();
lcd.setCursor(0, 0);
lcd.print("RFID SYSTEM");
lcd.setCursor(0, 1);
lcd.print("Scan Your Card");
Serial.println("LCD Ready");
Serial.println("RC522 Ready");
Serial.println("ThingSpeak Ready");
Serial.println("Buzzer Ready");
Serial.println("Scan Your Card");
Serial.println("=====");
}

// =====

// LOOP

```

```

// =====

void loop() {

    // -----

    // CHECK FOR NEW RFID CARD

    // -----

    if (!mfrc522.PICC_IsNewCardPresent()) {

        return;

    }

    // -----

    // READ RFID CARD

    // -----

    if (!mfrc522.PICC_ReadCardSerial()) {

        return;

    }

    // -----

    // CREATE UID STRING

    // -----

    String uid = "";

    Serial.print("UID: ");

    for (byte i = 0; i < mfrc522.uid.size; i++) {

        if (mfrc522.uid.uidByte[i] < 0x10) {

            Serial.print("0");

            uid += "0";

        }

        Serial.print(

            mfrc522.uid.uidByte[i],

            HEX

        );

        uid += String(

```

```
        mfr522.uid.uidByte[i],
        HEX
    );
}
uid.toUpperCase();
Serial.println();

// =====

// DOOR

// =====

if (uid == "BA8A8132") {
    showKnownTag(
        "Door",
        uid,
        1
    );
}

// =====

// LAB

// =====

else if (uid == "7BABD13E") {
    showKnownTag(
        "Lab",
        uid,
        2
    );
}

// =====

// SATHEESH

// =====
```

```
else if (uid == "D61A9269") {

    showKnownTag(
        "Satheesh",
        uid,
        3
    );
}
// =====

// SABEER
// =====

else if (uid == "6F22CB48") {

    showKnownTag(
        "Sabeer",
        uid,
        4
    );
}
// =====

// PRASANA
// =====

else if (uid == "1F22EB48") {

    showKnownTag(
        "Prasana",
        uid,
        5
    );
}
// =====
```

```

// UNKNOWN CARD

// =====

else {
    showUnknownTag(uid);
}

// -----

// STOP RFID COMMUNICATION

// -----

mfrc522.PICC_HaltA();
mfrc522.PCD_StopCrypto1();

// -----

// READY FOR NEXT CARD

// -----

delay(1000);

lcd.clear();

lcd.setCursor(0, 0);

lcd.print("RFID SYSTEM");

lcd.setCursor(0, 1);

lcd.print("Scan Your Card");
}

// =====

// FUNCTION: KNOWN RFID TAG

// =====

void showKnownTag(
    String name,
    String uid,
    int personID
) {
    // -----

```

```
// BUZZER OFF

// LOW ACTIVE BUZZER

// HIGH = OFF

// -----

digitalWrite(

    BUZZER_PIN,

    HIGH

);

// -----

// SERIAL OUTPUT

// -----

Serial.println();

Serial.print("Known RFID Tag: ");

Serial.println(name);

Serial.print("UID: ");

Serial.println(uid);

Serial.println("Access Granted");

Serial.println("-----");

// -----

// LCD OUTPUT

// -----

lcd.clear();

lcd.setCursor(0, 0);

lcd.print("Access Granted");

lcd.setCursor(0, 1);

lcd.print(name);

// -----

// SEND DATA TO THINGSPEAK

// -----
```

```

sendToThingSpeak(
    uid,
    personID,
    1
);
// -----

// KEEP RESULT ON LCD
// -----

delay(3000);
// -----

// MAKE SURE BUZZER IS OFF
// -----

digitalWrite(
    BUZZER_PIN,
    HIGH
);
}
// =====

// FUNCTION: UNKNOWN RFID TAG
// =====

void showUnknownTag(
    String uid
) {
    // -----

    // SERIAL OUTPUT
    // -----

    Serial.println();
    Serial.print("Unknown RFID Tag: ");
    Serial.println(uid);

```

```
Serial.println("Access Denied");

Serial.println("-----");

// -----

// LCD OUTPUT

// -----

lcd.clear();

lcd.setCursor(0, 0);

lcd.print("Unknown Tag");

lcd.setCursor(0, 1);

lcd.print("Access Denied");

// -----

// SEND DATA TO THINGSPEAK

// -----

sendToThingSpeak(

    uid,

    0,

    0

);

// -----

// BUZZER - LOW ACTIVE

// 3 BEEPS

// -----

for (int i = 0; i < 3; i++) {

    // BUZZER ON

    digitalWrite(

        BUZZER_PIN,

        LOW

    );

    delay(300);
```



```

// BUZZER OFF

digitalWrite(
    BUZZER_PIN,
    HIGH
);
delay(300);
}

// -----

// MAKE SURE BUZZER IS OFF

// -----

digitalWrite(
    BUZZER_PIN,
    HIGH
);

// -----

// KEEP RESULT ON LCD

// -----

delay(2000);
}

// =====

// FUNCTION: SEND DATA TO THINGSPEAK

// =====

void sendToThingSpeak(
    String uid,
    int personID,
    int accessStatus
) {
    // -----

    // CHECK WIFI

```

```
// -----  
if (WiFi.status() != WL_CONNECTED) {  
    Serial.println(  
        "WiFi Disconnected"  
    );  
    return;  
}  
  
// -----  
// FIELD 1 = RFID UID  
// -----  
ThingSpeak.setField(  
    1,  
    uid  
);  
  
// -----  
// FIELD 2 = PERSON ID  
// -----  
ThingSpeak.setField(  
    2,  
    personID  
);  
  
// -----  
// FIELD 3 = ACCESS STATUS  
// -----  
ThingSpeak.setField(  
    3,  
    accessStatus  
);  
  
// -----
```

```

// WRITE DATA TO THINGSPEAK
// -----
int response = ThingSpeak.writeFields(
    channelID,
    writeAPIKey
);
// -----
// CHECK RESPONSE
// -----
if (response == 200) {
    Serial.println(
        "ThingSpeak Update Successful"
    );
}
else {
    Serial.print(
        "ThingSpeak Error: "
    );
    Serial.println(response);
}
}

```

System Working:

- The Smart Inventory Management System uses an ESP32 microcontroller, RFID reader, load cell with HX711, OLED display and buzzer to monitor construction materials and track their movement.
- First, the RFID reader identifies the material using its unique RFID tag. The ESP32 receives the material identification and retrieves the corresponding material information. The load cell measures the weight or quantity of the material being added or removed.

- The user selects the required operation using the ADD or REMOVE buttons. Based on the selected operation, the ESP32 updates the inventory quantity accordingly. The updated inventory information is displayed on the OLED display for local monitoring.
- The ESP32 can also transmit the inventory data through Wi-Fi to a cloud database, allowing the site manager to monitor material availability remotely. If the inventory level falls below a predefined minimum limit, the buzzer is activated to provide a low-stock alert.
- The system continuously repeats the process, enabling real-time tracking of material quantity and movement. This helps reduce material wastage, prevent stock shortages and improve resource utilization and construction site management efficiency.

Bill Of Material:

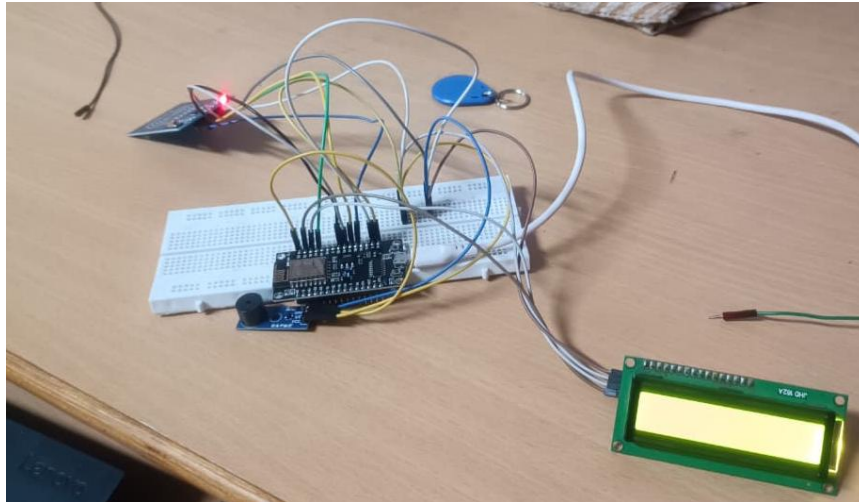
S. no	Component	Specification	Quantity	Cost
1	ESP8266	Node MCU ESP8266	1	Rs. 450
2	RFID Reader	RC522 RFID Module	1	Rs. 150
3	OLED Display	0.96" I ² C, SSD1306, 128×64	1	Rs. 150
5	Breadboard	Standard 830-point	1	Rs. 100
6	Jumper Wire	Male-Male/Male-Female	1 set	Rs. 50
			Total	Rs. 900

Prototype Implementation:

- The prototype was implemented using an ESP32 microcontroller, RFID RC522 reader, load cell with HX711 module, OLED display, push buttons and buzzer. The RFID reader is used to identify construction materials through unique RFID tags, while the load cell measures the weight of the material being added or removed.
- The ESP32 processes the RFID and weight data and updates the inventory quantity based on the operation selected using the ADD and REMOVE buttons. The current

inventory status is displayed on the OLED display for easy monitoring at the construction site.

- A buzzer is incorporated to provide alerts when an invalid RFID tag is detected or the material quantity falls below the predefined minimum stock level. The ESP32's built-in Wi-Fi capability can also be used to transmit inventory information to a cloud database for remote monitoring and data analysis.



Results and Performance:

- The developed **Smart Inventory Management System** successfully monitored and tracked construction materials using the **ESP32, RFID reader, load cell and HX711 module**. The RFID system identified the materials, while the load cell measured their weight during inventory operations.
- The ESP32 accurately processed the collected information and updated the inventory quantity based on whether material was **added or removed**. The updated inventory status was displayed on the OLED display, providing real-time information to the user.
- The buzzer successfully provided an alert when an **invalid RFID tag** was detected or when the inventory level fell below the predefined minimum limit. The system can also transmit inventory information through Wi-Fi for remote monitoring and cloud-based storage.
- Overall, the prototype demonstrated a low-cost, automated and efficient approach to construction inventory management. It reduces dependence on manual stock recording and helps improve material tracking, resource utilization and stock management. The system can be further enhanced with a web/mobile dashboard, multiple RFID readers, improved weighing accuracy and automated material replenishment alerts.

